

Cultura Viva

An on-device multimodal audio guide for Gaudí’s Barcelona
Model selection, benchmarks, and bill of materials

Cultura Viva team

13 September 2026

Abstract

Cultura Viva is an interactive multimodal AI audio guide for Antoni Gaudí’s monuments in Barcelona — Sagrada Família, Park Güell, Casa Milà (La Pedrera) and Casa Batlló — running *entirely on-device* on an Arduino UNO Q. A visitor photographs an architectural element, asks a question aloud, and hears a spoken answer in the guide personality they picked. No network round-trip is involved at inference time.

This report documents the engineering work behind the device: the four-stage pipeline (vision $\rightarrow$ speech-to-text $\rightarrow$ knowledge retrieval + small language model $\rightarrow$ speech synthesis), the comparative benchmarks that selected each model under the board’s memory and latency budget, the latency optimisations that make the interaction feel immediate, and the bill of materials with per-component prices.

Contents

1	System overview	3
1.1	Target platform	3
1.2	The pipeline	3
1.3	Design constraints	3
2	Vision models	4
2.1	Problem formulation	4
2.2	Dataset and splits	4
2.3	Backbones tried	4
2.4	Results	5
2.5	Out-of-distribution rejection	5
2.6	Export path	6
3	Benchmarks	6
3.1	Small language model	6
3.1.1	Candidates	6
3.1.2	Protocol	6
3.1.3	Results	6
3.2	Speech-to-text	7
3.2.1	Engines compared	7
3.2.2	Metrics and optimisations	7
3.2.3	Results	8
3.3	Text-to-speech	8
3.3.1	Candidates	8
3.3.2	Protocol	9
3.3.3	Reading the results	9
3.4	End-to-end latency	10

4	Bill of materials	10
4.1	Arduino components	10
4.2	Third-party components	11
4.3	Software cost	11
5	Summary of selections	12
6	Open items	12

1 System overview

1.1 Target platform

The whole system runs on a single Arduino UNO Q, which is a dual-brain board:

- **STM32 microcontroller (MCU).** Real-time C++ Arduino sketch driving the ST7735S LCD, the physical inputs (mode switch, action button, Modulino buttons/knob), the Modulino buzzer, GPS ingestion and minimap rendering.
- **Qualcomm QRB2210 Linux processor (MPU).** Quad-core ARM Cortex-A53 @ 2.0 GHz, LPDDR4 RAM, 32 GB eMMC, Debian Linux. Runs the Python AI pipeline: ONNX vision classification, faster-whisper STT, knowledge-graph retrieval, a quantised small language model via `llama-cpp-python`, and Piper TTS.

The two halves talk over Arduino Bridge RPC. The board variant used for all measurements reports 3667 MB of usable RAM, i.e. the 4 GB model.

1.2 The pipeline

1. **Capture & validate.** The visitor frames a shot in camera mode (live view streamed to the LCD as Base64 RGB565 chunks over Bridge RPC) and presses the shutter. A per-monument ONNX classifier decides *which architectural element* is in frame, or rejects the photo as off-distribution and asks for a retake.
2. **Ask.** The visitor flips to map mode, picks a guide personality on the Modulino buttons (Artistic / Technical / Child), and records a spoken question. Recording ends by itself on silence.
3. **Understand.** faster-whisper transcribes the question, biased towards the domain vocabulary (“Gaudí”, “trencadís”, “Sagrada Família”, ...) through an `initial_prompt`.
4. **Ground & answer.** The recognised element ID indexes directly into a curated knowledge base (`knowledge_base.json`, `element_sheets.json`), with keyword search as fallback. The retrieved facts plus the personality prompt plus the question go to the SLM.
5. **Speak.** Piper synthesises the answer sentence by sentence, streamed to ALSA while the SLM is still decoding the next sentence. Volume tracks the Modulino knob live.
6. **Map.** The visited landmark is marked on the LCD minimap, with position from the NEO-6M GPS.

1.3 Design constraints

Every model choice in this report was made against the same budget:

Constraint	Value
Compute	4× ARM Cortex-A53 @ 2.0 GHz, CPU only
RAM for SLM weights + context	< 1.5 GB
SLM context window	2048 tokens
Target response latency	< 3 s to first audio
Connectivity at inference time	none (fully offline)

There is no GPU backend: the `llama.cpp` build on the board is CPU-only, and ONNX Runtime uses the CPU execution provider.

2 Vision models

2.1 Problem formulation

The vision stage answers “*what* is the visitor pointing at?”. It is a per-monument classification problem, not a single global one: each monument has its own set of architectural elements, so the label spaces are disjoint and a shared classification head would be meaningless. We therefore train one classifier per (monument, model) pair.

Table 1: Element classes per monument. Class counts differ freely; nothing in the pipeline assumes they match.

Monument	N	Classes
Casa Batlló	2	pla_frontal, pla_inferior
Park Güell	7	3_viaductes, casa_museu, escalinata_drac, pavellons_consergeria, placa_natura, sala_hipostila, turo_3_creus
La Pedrera (Casa Milà)	3	facana_1, facana_2, facana_3
Sagrada Família	6	cupula, facana_naixement, facana_passio, laterals, posterior, torres

Classes are auto-detected from each monument’s own subfolders in the dataset, so adding a monument or a class requires no code change.

2.2 Dataset and splits

All images come from the culturaviva/image_1080 Hugging Face dataset, one subfolder per monument. The split is fixed and shared by training, validation and threshold calibration:

Train / validation / test	70 % / 15 % / 15 %
Split seed	42
Input resolution	224×224
Normalisation	mean 0.5, std 0.5 per channel

Training and calibration both go through the same dataset loader, which guarantees that OOD thresholds are calibrated on images the model was never fit to.

2.3 Backbones tried

Table 2: Vision backbones. A third backbone, ResNet-50 (microsoft/resnet-50), is present in the configuration but currently commented out and was not part of the reported runs.

Name	Hugging Face checkpoint	Status
mobilenetv2	google/mobilenet_v2_1.0_224	active
vit	google/vit-base-patch16-224	active
resnet50	microsoft/resnet-50	configured, disabled

With $4 \text{ monuments} \times 2 \text{ active backbones}$, a full pass is **8 training runs**. Each run gets its own dataset split instance, model instance, output directory and Weights & Biases run, named `<monument>-<model>`, so runs never overwrite each other.

Training hyper-parameters. Applied to every run unless a model entry overrides them:

Epochs	8
Batch size	16
Learning rate	2×10^{-5}
Model selection	best checkpoint by validation macro-F1

Evaluation reports **accuracy** and **macro-F1** (macro so that rare elements are not drowned out by the frequent ones), computed on the held-out test split at the end of each run.

2.4 Results

Table 3: Test-split accuracy and macro-F1 per (monument, backbone) run. *These values are logged to the Weights & Biases project `cultura-viva` and were not exported into the repository; the placeholders below are to be filled from the corresponding W&B runs.*

Monument	N	MobileNetV2		ViT-B/16	
		Acc.	macro-F1	Acc.	macro-F1
Casa Batlló	2	[TBD]	[TBD]	[TBD]	[TBD]
Park Güell	7	[TBD]	[TBD]	[TBD]	[TBD]
La Pedrera (Casa Milà)	3	[TBD]	[TBD]	[TBD]	[TBD]
Sagrada Família	6	[TBD]	[TBD]	[TBD]	[TBD]
Mean		[TBD]	[TBD]	[TBD]	[TBD]

What ships. Two exported classifiers are currently deployed on the device, under `models/vision/<location>/`: Park Güell (7 element classes + `unknown`) and Sagrada Família (6 element classes + `unknown`). Each is a `model.onnx` plus a `labels.json` carrying the label map, input size and normalisation constants, so the board-side code needs no per-model configuration.

2.5 Out-of-distribution rejection

A real visitor will point the camera at a tree, another tourist, or the sky. A plain `argmax` classifier is obliged to name an architectural element anyway, which would produce a confident, grounded, entirely wrong audio guide answer. The pipeline therefore puts a gate in front of the prediction. The input is rejected as `not_sure` when *either* signal fires:

$$H(p) = - \sum_{i=1}^N p_i \ln p_i > \tau_H \quad (\text{predictive entropy too high}) \quad (1)$$

$$\max_i p_i < \tau_{\text{conf}} \quad (\text{peak confidence too low}) \quad (2)$$

Defaults are $\tau_H = 0.5 \ln N$ — auto-scaled to each monument’s class count, so a 2-class and a 7-class head are held to comparable standards — and $\tau_{\text{conf}} = 0.50$.

Thresholds merge across three levels, `global` $\rightarrow$ `monument` $\rightarrow$ `model`, with an optional CLI override on top; the most specific setting wins. This matters because the backbones do not calibrate alike: MobileNetV2 tends to run less confident than ViT, and can be loosened on its own without touching the other heads.

`calibrate_thresholds.py` sweeps the validation split and suggests $(\tau_H, \tau_{\text{conf}})$ for a given checkpoint, subject to a *recall target* — how much in-distribution recall must be preserved, default 95%. It prints a ready-to-paste YAML snippet.

2.6 Export path

1. `train_classifier.py` — fine-tunes every (monument, model) combination in `config.yaml`.
2. `export_to_onnx.py` — converts one fine-tuned checkpoint into `model.onnx` + `labels.json`.
3. `calibrate_thresholds.py` — suggests the OOD thresholds.
4. `inference.py` — runs an exported checkpoint on a single image with the gate applied; also importable as `MonumentClassifier`, which is how the device uses it.

3 Benchmarks

Three separate benchmark suites were built, one per non-vision stage. All three target the same hardware; where a measurement was taken on a host PC rather than on the board, this is stated explicitly.

3.1 Small language model

3.1.1 Candidates

Five instruction-tuned small models were evaluated, all at Q4_K_M quantisation so that weights plus a 2k context fit the < 1.5 GB budget.

Table 4: SLM candidates. RAM footprint is weights plus 2k context.

Model	Ollama tag	Weights	RAM
Qwen 2.5 1.5B Instruct	<code>qwen2.5:1.5b</code>	1.10 GB	1.45 GB
Llama 3.2 1B Instruct	<code>llama3.2:1b</code>	0.80 GB	1.10 GB
Gemma 3 1B IT	<code>gemma3:1b</code>	0.85 GB	1.15 GB
SmolLM2 1.7B Instruct	<code>smollm2:1.7b</code>	1.15 GB	1.50 GB
Qwen 2.5 0.5B Instruct	<code>qwen2.5:0.5b</code>	0.39 GB	0.65 GB

3.1.2 Protocol

- **Test set.** 38 curated Gaudí questions with reference answers and retrieved contexts (`eval/testset.json`).
- **Generation.** `temperature` = 0.1, `max_tokens` = 128, context window 2048, 4 threads, batch size 256.
- **Judge.** Ragas, scoring *Faithfulness*, *AnswerRelevancy*, *ContextPrecision* and *ContextRecall*, driven by a *local* Ollama judge (`qwen2.5:7b`, temperature 0) with `all-MiniLM-L6-v2` sentence embeddings. Keeping the judge local means the evaluation is reproducible offline and no test data leaves the machine.
- **On-device re-run.** `benchmark_arduino.py` re-runs the same 38 questions on the Cortex-A53 through `llama-cpp-python` built with `-march=armv8-a -mtune=cortex-a53`, and the resulting predictions are scored with the same Ragas judge back on the PC.

3.1.3 Results

The spread in generation time is small — roughly 15% between the fastest and the slowest candidate — which is the important negative result: on this workload, *latency does not discriminate between the candidates*, so the choice can be made on answer quality and memory headroom instead.

Table 5: Per-answer generation wall-clock over the 38-question test set, as recorded in the checked-in prediction files. These runs were produced through Ollama on the development host; they are *not* Cortex-A53 timings and should be read as a relative ordering between candidates, not as device latency.

Model	Mean (s)	Median (s)	Min (s)	Max (s)
Llama 3.2 1B Instruct	2.42	2.40	2.10	4.33
Qwen 2.5 0.5B Instruct	2.43	2.41	2.09	3.59
Gemma 3 1B IT	2.57	2.48	2.17	5.52
Qwen 2.5 1.5B Instruct	2.61	2.61	2.14	4.97
SmolLM2 1.7B Instruct	2.79	2.65	2.18	5.96

Table 6: Ragas scores over the 38-question test set. *The scored results (eval/results/) are not checked into the repository; re-run main.py benchmark to regenerate.*

Model	Faithful.	Ans. Rel.	Ctx. Prec.	Ctx. Rec.
Qwen 2.5 1.5B Instruct	[TBD]	[TBD]	[TBD]	[TBD]
Llama 3.2 1B Instruct	[TBD]	[TBD]	[TBD]	[TBD]
Gemma 3 1B IT	[TBD]	[TBD]	[TBD]	[TBD]
SmolLM2 1.7B Instruct	[TBD]	[TBD]	[TBD]	[TBD]
Qwen 2.5 0.5B Instruct	[TBD]	[TBD]	[TBD]	[TBD]

Selected. **Qwen 2.5 1.5B Instruct**, Q4_K_M (qwen2.5-1.5b-instruct-q4_k_m.gguf), is the configured default for Arduino deployment. Qwen 2.5 0.5B remains a supported fallback for the 2 GB board variant, where the 1.5B model’s 1.45 GB working set is uncomfortably tight.

3.2 Speech-to-text

3.2.1 Engines compared

Table 7: STT engines evaluated, all on the Whisper Base model family.

Engine	Runtime	Status
faster-whisper	CTranslate2, INT8 quantisation	primary
whisper.cpp	native C++ / GGML, ARM NEON SIMD	primary
vosk	Kaldi-derived	optional
sherpa-onnx	ONNX Runtime	optional

For `whisper.cpp`, `pywhispercpp`’s bundled backend is rebuilt against the target core:

```
export CFLAGS="-O3 -mcpu=cortex-a53"
export CXXFLAGS="-O3 -mcpu=cortex-a53"
uv pip install -force-reinstall pywhispercpp>=1.2.0
```

On 64-bit aarch64 NEON is on by default; `-mcpu=cortex-a53` is about instruction scheduling for this specific core.

3.2.2 Metrics and optimisations

The harness measures **WER**, **CER**, **keyword-spotting accuracy**, **real-time factor** (inference time / audio duration) and **peak RAM**.

Keyword-spotting accuracy deserves a note. A generic WER number hides the failure that actually matters for an audio guide: a transcript that gets “Gaudí”, “trencadís” or “Sagrada Família” wrong will retrieve the wrong knowledge sheet, and the whole answer is then confidently off-topic. The decoder is therefore biased towards the domain vocabulary through an `initial_prompt`:

Cultura Viva audio guide in Barcelona about Antoni Gaudí, Sagrada Família basilica, Nativity, Passion, and Glory facades, Catalan modernisme architecture, Casa Batlló, Casa Milà, Park Güell, dragon and salamander sculptures, and trencadís mosaics.

For **faster-whisper** the same terms are additionally passed as explicit **hotwords**. Further tuning:

- **4 CPU threads** — the QRB2210 has exactly 4 Cortex-A53 cores, so 4 saturates the CPU without paying context-switch overhead.
- **INT8 compute type** — keeps the model comfortably inside LPDDR4 and maximises memory bandwidth.
- **Greedy decoding** — `temperature = 0.0`, `beam_size = 1`, to minimise RTF.

3.2.3 Results

The evaluation set is **24 synthetic visitor utterances**, varied across register (neutral, polite, child-like), intent (materials, history, ...) and speaker voice, with controlled background noise, each annotated with its target keywords.

Table 8: STT results over the 24-utterance set, domain bias applied. Timings are from the `results_computer/run`. *Only faster-whisper:base.en results are checked into the repository; the remaining engines’ rows are to be filled from a re-run.*

Engine	WER	CER	Keyword acc.	Peak RAM	Model
<code>faster-whisper:base.en</code>	0.164	0.093	96.1 %	397 MB	141 MB
<code>whisper.cpp:base.en-q5_1</code>	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
<code>vosk</code>	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]
<code>sherpa-onnx</code>	[TBD]	[TBD]	[TBD]	[TBD]	[TBD]

A 16.4 % word error rate alongside 96.1 % keyword accuracy is exactly the shape we were optimising for: the transcript is imperfect in its function words but almost never wrong about the proper nouns that drive retrieval.

Selected. **faster-whisper** with the `base.en` model at INT8, which is what the device ships (`models/stt/faster-whisper-base.en`).

3.3 Text-to-speech

3.3.1 Candidates

18 ONNX voices are registered in `models_config.json`: three VITS models exported from Hugging Face (MMS-TTS English, VITS LJSpeech, VITS VCTK) and 15 Piper voices (Lessac, Ryan, Amy, Alan, Cori, each in low / medium / high quality). 15 of the 18 were actually timed; `piper-amy-high`, `piper-alan-high` and `piper-cori-low` were not run.

Audio quality is explicitly out of scope for this benchmark — it measures *inference time and footprint only*. Voice character was chosen separately, by ear, per guide personality.

3.3.2 Protocol

Measured on the UNO Q itself (Qualcomm QRB2210, Cortex-A53, 4 threads, 3667 MB RAM detected), 2026-08-26. Each model is loaded once, given 1 untimed warm-up run, then timed over **9 runs** round-robin balanced across sentence-length buckets — short (≤ 8 words), medium (9–16), long (≥ 17) — drawn from a fixed set of Gaudí sentences.

Table 9: TTS inference time on the Arduino UNO Q. Median seconds per sentence by length bucket; RTF is the median of (inference time / generated audio duration), so $\text{RTF} < 1$ means the voice synthesises faster than real time. Disk size is the configured on-disk model size.

Voice	Disk (MB)	Short (s)	Medium (s)	Long (s)	RTF
<i>VITS (ONNX Runtime, CPU)</i>					
MMS-TTS (English, VITS, ONNX)	138	12.91	21.95	40.92	3.88
VITS (LJSpeech, ONNX)	332	15.39	43.89	59.04	3.78
VITS (VCTK, multi-speaker, ONNX)	150	13.69	34.43	49.50	3.80
<i>Piper</i>					
Piper (en_US-lessac-low)	20	1.61	4.12	6.00	0.62
Piper (en_US-lessac-medium)	63	1.91	4.91	7.28	0.77
Piper (en_US-lessac-high)	113	12.86	33.82	50.45	5.19
Piper (en_US-ryan-low)	20	1.57	3.89	5.87	0.63
Piper (en_US-ryan-medium)	63	1.94	5.36	7.43	0.80
Piper (en_US-ryan-high)	113	12.52	37.24	51.76	5.31
Piper (en_US-amy-low)	20	1.79	4.28	5.95	0.61
Piper (en_US-amy-medium)	63	2.44	5.83	8.23	0.80
Piper (en_GB-alan-low)	20	1.85	5.07	7.30	0.64
Piper (en_GB-alan-medium)	63	2.39	6.14	8.67	0.75
Piper (en_GB-cori-medium)	63	2.27	4.51	7.19	0.81
Piper (en_GB-cori-high)	113	13.31	31.94	52.32	5.27

3.3.3 Reading the results

Table 9 separates the field cleanly into three groups.

- **The VITS exports are unusable here.** $\text{RTF} \approx 3.8$ means roughly four seconds of compute per second of speech. A 20-second answer would take over a minute to synthesise.
- **Piper “high” voices are equally unusable**, and for the same reason — $\text{RTF} \approx 5.2\text{--}5.3$, *worse* than the VITS models despite Piper’s much lighter runtime. The quality tier, not the framework, is what blows the budget.
- **Piper low and medium clear the bar with room to spare.** Every low voice lands at $\text{RTF} \approx 0.61\text{--}0.64$ and every medium voice at $\approx 0.75\text{--}0.81$. A medium voice synthesises a short sentence in about 2 s and a long one in about 8 s, while producing more audio than that in playback time — which is what makes sentence-level streaming (§3.4) work: playback of sentence N outlasts synthesis of sentence $N + 1$, so the audio never stalls.

The gap between low and medium is around 25 % of compute for a clearly better voice, which is worth paying given the headroom.

Selected. Two Piper medium voices ship on the device, mapped to guide personalities:

Personality	Style	Voice
A — Artistic	Evocative, metaphorical	en_US-libritts_r-medium
B — Technical	Architectural, precise	en_GB-semaine-medium (Spike)
C — Child	Simple, fun, story-led	en_GB-semaine-medium (Prudence)

3.4 End-to-end latency

`python/benchmark.py` on the device measures the stages in isolation and then reports the figure that actually matters.

Why the serial sum is the wrong number. Adding up every stage measured in isolation gives an upper bound, not a latency, because the pipeline deliberately does not run them in sequence: vision happens at photo time, prefill overlaps with recording, and Piper synthesises while `llama.cpp` is still decoding.

The benchmark therefore reports **time to first audio** — from the moment the visitor stops speaking to the first word they hear. It counts only the stages still on that path, each with the figure that applies there: STT in full, the *warm* prefill rather than the cold one, decode only as far as the first complete sentence, and one sentence of Piper rather than the whole answer. Any stage that could not be measured is named and the total flagged as an underestimate, rather than being silently dropped. It excludes the `aplay` spawn and Bridge RPC round-trips, which the harness does not exercise, so the board is somewhat slower than the printed figure.

The three optimisations.

1. **Asynchronous prefill overlap.** Once a photo is confirmed, the static prompt prefix — system instructions plus the monument’s fact sheet — is prefilled into the SLM KV-cache on a background thread while the visitor is still speaking. When recording ends, only the new user question needs prefilling. The prefix cache is additionally persisted to disk: `prewarm_cache.py` computes it once per element (≈ 30 s each, ≈ 12 minutes for all 24), so no visitor ever sits through a cold ≈ 30 s prefill.
2. **RMS silence detection with cropping.** Recording ends by itself once the visitor stops talking (≈ 1.5 s of silence), and the trailing and leading silence is cropped before the samples reach Whisper. This is not cosmetic: Whisper’s encoder pads every window to a fixed length, so room tone between the last word and the stop is encoded at exactly the same price as speech. Cropping removes that cost from the critical path entirely. Thresholds adapt to the noise floor measured at the start of each recording; in a room too loud to judge, detection backs off and the physical button stops the recording as before.
3. **Sentence-level streaming to TTS.** Rather than waiting for the full generation (60+ tokens), tokens are emitted sentence by sentence. Piper synthesises sentence N while the SLM decodes sentence $N + 1$. Given the Piper medium RTF of ≈ 0.8 measured above, the synthesis pipeline stays comfortably ahead of playback.

Together these move the perceived latency from “sum of all four stages” to “STT + warm prefill + one sentence of decode + one sentence of Piper”.

4 Bill of materials

4.1 Arduino components

Prices are Arduino Store (EU) list prices, VAT included, retrieved 13 September 2026 from <https://store.arduino.cc>. Product codes are given where available.

Table 10: Arduino components. The three Modulino modules are the ones this project actually requires.

Component	Code	Qty	Role in Cultura Viva	Price
Arduino UNO Q 4GB	—	1	MCU + Linux MPU, the whole system	82.90 €
<i>Modulino modules (I²C, Qwiic / Wire1)</i>				
Modulino Buttons	ABX00110	1	Guide personality select A/B/C + LEDs	8.50 €
Modulino Knob	ABX00107	1	Live headphone volume 0–100 %	9.00 €
Modulino Buzzer	ABX00108	1	Shutter click, confirmations, alerts	7.30 €
Qwiic Cables Bundle	—	1	Chains the Modulinos to the Wire1 port	10.60 €
Modulino subtotal				24.80 €
Arduino subtotal				118.30 €

Note on availability. At the time of retrieval the Modulino Knob (ABX00107) was listed as *sold out* on the Arduino EU store. Its price is as listed.

Note on the board variant. The UNO Q also exists in a 2 GB / 16 GB variant at a lower price. All measurements in this report were taken on the 4 GB board (3667 MB detected). The 2 GB variant would run the pipeline only with the Qwen 2.5 0.5B fallback SLM, since the 1.5B model’s ≈ 1.45 GB working set leaves no room.

4.2 Third-party components

Table 11: Non-Arduino components. *These prices are indicative street prices and were not verified against a single named vendor at the time of writing — replace with your actual purchase prices before publishing.*

Component	Interface	Qty	Role	Price
ST7735S LCD, 1.8 " 128×160	SPI	1	Live view, minimap, UI	[TBD]
Logitech Brio 105 webcam	USB 2.0	1	1080p capture + microphone	[TBD]
NEO-6M GPS module	Serial1 @ 9600	1	Monument proximity detection	[TBD]
Mode switch (SPDT)	Digital D6	1	Camera / map mode	[TBD]
Push button	Digital D7	1	Shutter / record / cancel	[TBD]
Headphones or speaker	3.5 mm jack	1	Spoken answer output	[TBD]
Jumper wires, carrier board	—	—	Assembly	[TBD]
Third-party subtotal				[TBD]

4.3 Software cost

Zero. Every model in the pipeline is open-weights and runs locally: the vision backbones (google/mobilenet_v2_1.0_224, google/vit-base-patch16-224), Whisper Base via faster-whisper, Qwen 2.5 1.5B Instruct at Q4_K_M, and the Piper voices from rhasspy/piper-voices. There are no inference API calls and no per-visitor marginal cost. The Ragas judge used during evaluation (qwen2.5:7b through Ollama) also runs locally and is a development-time dependency only — it never ships to the device.

5 Summary of selections

Table 12: What the benchmarks selected, and on what grounds.

Stage	Selected	Decided on
Vision	Per-monument ONNX classifiers, 224×224 , with entropy + confidence OOD gate	Accuracy / macro-F1 per monument; OOD rejection is a hard requirement, not a nicety
STT	faster-whisper <code>base.en</code> , INT8, 4 threads, greedy	16.4 % WER but 96.1 % keyword accuracy — proper nouns drive retrieval
SLM	Qwen 2.5 1.5B Instruct Q4_K_M (0.5B fallback for the 2 GB board)	Answer quality and memory headroom; latency did not discriminate between candidates
TTS	Piper medium voices, per personality	RTF ≈ 0.8 on the board, which is what makes sentence-level streaming viable

6 Open items

The following numbers are referenced in this report but are not currently exported into the repository, and appear above as **[TBD]**:

1. **Vision accuracy / macro-F1** for the 8 (monument, backbone) runs (Table 3). Logged to the Weights & Biases project `cultura-viva`; `vision/outputs/` is gitignored.
2. **Ragas scores** for the 5 SLM candidates. `eval/results/` is not checked in; regenerate with `uv run python main.py benchmark`.
3. **STT results for whisper.cpp, Vosk and sherpa-onnx**. Only `faster-whisper:base.en` is present in `results_computer/summary.json`.
4. **Third-party component prices**.
5. **Measured time-to-first-audio** from a board run of `python/benchmark.py -json`.

TTS inference time by sentence length, vs. model size

Arduino UNO Q (Qualcomm QRB2210, Cortex-A53, Linux) · median $\pm$ min/max per length bucket, over Gaudí-related sentences

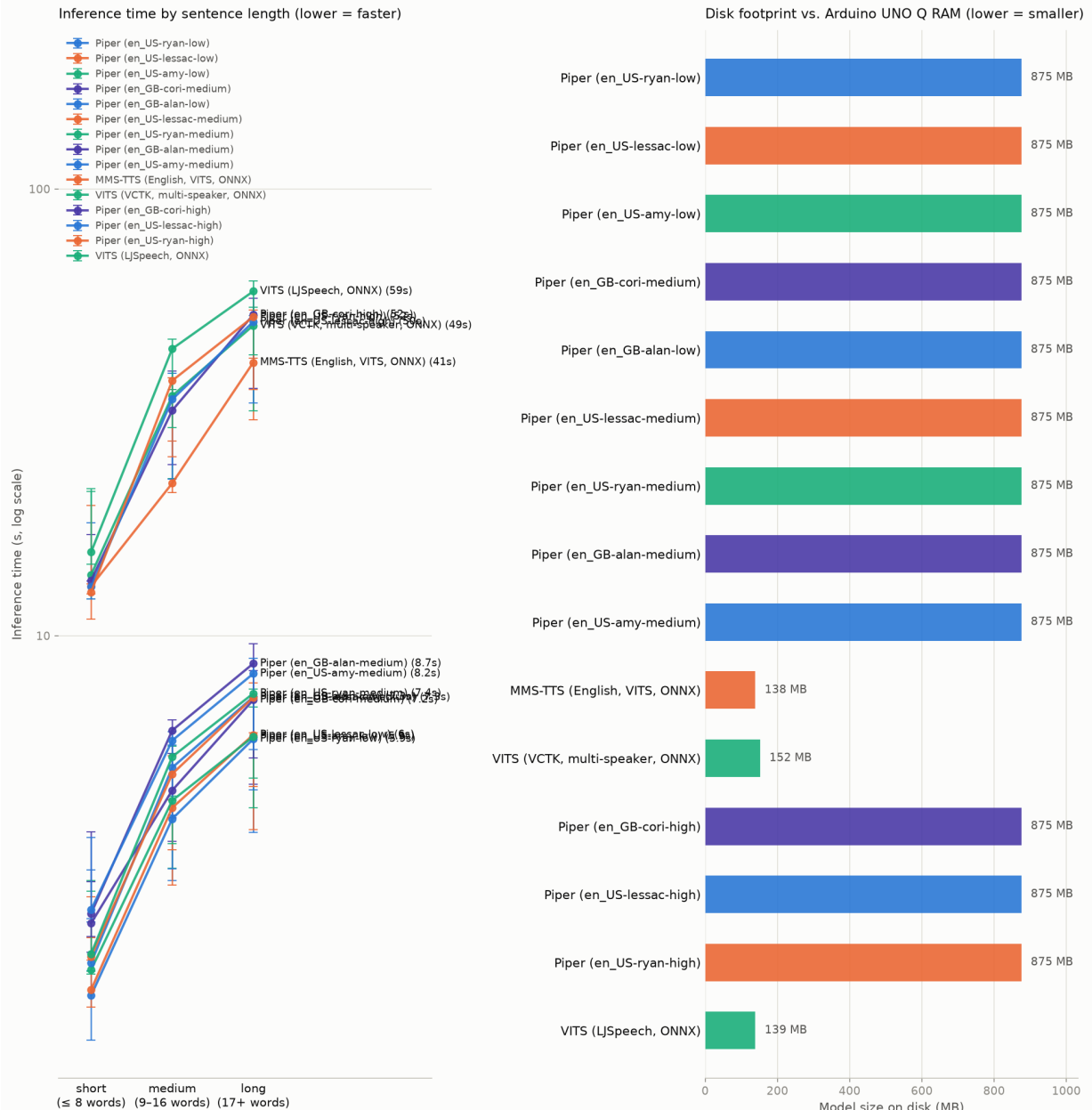


Figure 1: Median inference time with min/max whiskers per sentence-length bucket, alongside on-disk model size, generated by `visualize.py` from the same results file as Table 9.